

CSS for Beginners

Complete Notes

Classroom Study Material · Exam & Interview Revision

Topics Covered

CSS Syntax · Selectors · Colors · Text Styling
CSS Units · Box Model · Display Property
Flexbox · **CSS Grid** · Positioning
Pseudo-classes · Transitions · Best Practices

Web Development Course — Module 2

Study this material carefully.

Every section contains examples you can type and run yourself.

“CSS makes the web beautiful.

Master the Box Model, Flexbox, and Grid — and you can build any layout.”

Contents

1	Introduction to CSS	3
1.1	What is CSS?	3
1.2	Why is CSS Used in Web Development?	3
1.3	How CSS Works with HTML	3
2	CSS Fundamentals	5
2.1	CSS Syntax	5
2.2	Ways to Add CSS	5
2.2.1	1. Inline CSS	6
2.2.2	2. Internal CSS	6
2.2.3	3. External CSS (MOST IMPORTANT — Recommended)	6
2.3	CSS Selectors (Very Important)	7
2.3.1	Element Selector	7
2.3.2	Class Selector	8
2.3.3	ID Selector	8
2.3.4	Group Selector	9
2.4	Colors in CSS	9
2.5	Text Styling	10
3	CSS Units of Measurement (Very Important Core Topic)	12
3.1	Absolute Units	12
3.1.1	px — Pixels (MOST IMPORTANT)	12
3.1.2	Other Absolute Units (Brief Reference)	12
3.2	Relative Units (Very Important)	12
3.2.1	% — Percentage	13
3.2.2	em — Relative to Parent Font Size	13
3.2.3	rem — Relative to Root Font Size (Very Important)	13
3.2.4	vh and vw — Viewport Units	14
3.3	When to Use Which Unit	15
4	The Box Model (Core Concept — Must Understand)	16
4.1	What is the Box Model?	16
4.2	The Four Parts Explained	16
4.3	Total Element Size	17
4.4	Shorthand for Padding and Margin	17
4.5	Display Property	18
5	Layout Systems	19
5.1	Flexbox (Must Learn — Modern CSS Essential)	19
5.1.1	What is Flexbox and Why Use It?	19
5.1.2	How Flexbox Works	19
5.1.3	Main Axis vs Cross Axis	19
5.1.4	justify-content — Align Along Main Axis	20
5.1.5	align-items — Align Along Cross Axis	20
5.1.6	Complete Flexbox Examples	20
5.2	CSS Grid (Must Learn — 2D Layout System)	22
5.2.1	What is CSS Grid and Why Use It?	22

5.2.2	Basic Grid Setup	22
5.2.3	grid-template-columns and grid-template-rows	22
5.2.4	Complete Grid Examples	23
5.3	Basic Layout Properties	24
6	Learn-As-You-Go Topics (Introduction Only)	26
6.1	Position Property	26
6.2	z-index	26
6.3	Pseudo-classes	27
6.4	Transitions	27
7	Common Mistakes to Avoid	29
8	Best Practices	30
9	Practice Exercises	31
9.1	Exercise 1: Center a Div Using Flexbox	31
9.2	Exercise 2: Create a Navbar Using Flexbox	32
9.3	Exercise 3: Create a Grid Photo Gallery	33
9.4	Exercise 4: Style a Profile Card	33
10	Quick Reference Summary	36
10.1	CSS Units Cheat Sheet	36
10.2	CSS Selectors Cheat Sheet	36
10.3	Flexbox Cheat Sheet	36
10.4	Grid Cheat Sheet	37
10.5	Box Model Cheat Sheet	37
11	Interview & Exam Revision Points	38
	Final Note to Students	39

1 Introduction to CSS

1.1 What is CSS?

CSS stands for **Cascading Style Sheets**. It is a language used to *style* the visual appearance of web pages.

If HTML is the *skeleton* of a webpage (structure and content), then CSS is the *skin and clothes* — it decides the colors, fonts, sizes, spacing, and layout of everything you see.

Simple Definition: HTML says “*Here is a heading.*” CSS says “*Make that heading blue, size 32px, and centered.*”

CSS is not a programming language. It is a **stylesheet language** — a set of rules that tell the browser how to display HTML elements.

1.2 Why is CSS Used in Web Development?

Without CSS, every webpage would look like plain black text on a white background — no colors, no layout, no design.

CSS lets you:

- Change colors, fonts, and text sizes
- Control spacing between elements (padding & margin)
- Build complex layouts (side-by-side columns, grids, navbars)
- Make websites responsive (look good on mobile & desktop)
- Add hover effects, transitions, and animations
- Keep design separate from content (clean code structure)

Key Benefit: One CSS file can style hundreds of HTML pages. Change the CSS file once — and the entire website updates automatically.

1.3 How CSS Works with HTML

CSS works by **selecting** an HTML element and then applying **style rules** to it.

Think of it as sending instructions to the browser: “*Find all <p> tags and make the text gray and 16px.*”

Here is a simple example of HTML + CSS working together:

```
1 <!-- index.html -->
2 <!DOCTYPE html>
3 <html>
4   <head>
5     <link rel="stylesheet" href="style.css">
```

```
6 </head>
7 <body>
8   <h1>Hello, World!</h1>
9   <p>This is a paragraph.</p>
10 </body>
11 </html>
```

Listing 1: HTML file linked to a CSS file

```
1 /* style.css */
2 h1 {
3   color: blue;
4   font-size: 36px;
5 }
6
7 p {
8   color: gray;
9   font-size: 16px;
10 }
```

Listing 2: style.css — styling the HTML

The browser reads both files and displays the heading in blue and the paragraph in gray.

2 CSS Fundamentals

2.1 CSS Syntax

Every CSS rule follows the same simple structure:

```
selector { property: value; }
```

- **Selector** — Targets the HTML element(s) you want to style (e.g., `h1`, `.card`, `#header`)
- **Property** — The style attribute you want to change (e.g., `color`, `font-size`)
- **Value** — The value you assign to that property (e.g., `red`, `24px`)
- Each rule ends with a **semicolon** (`;`)
- Rules are wrapped in **curly braces** (`{ }`)

```
1  /* This is a comment in CSS */
2
3  h1 {
4      color: navy;
5      font-size: 32px;
6      text-align: center;
7  }
8
9  p {
10     color: #555555;
11     font-size: 16px;
12     line-height: 1.6;
13 }
14
15 .highlight {
16     background-color: yellow;
17     font-weight: bold;
18 }
```

Listing 3: CSS Syntax — Full Example

Important: CSS is read top-to-bottom. If two rules target the same element, the *last one written* usually wins (this is the “cascading” part of CSS).

2.2 Ways to Add CSS

There are three ways to add CSS to an HTML page. You should know all three, but one is clearly the best.

2.2.1 1. Inline CSS

Written directly inside an HTML element using the `style` attribute.

```
1 <h1 style="color: red; font-size: 28px;">Hello</h1>
2 <p style="color: gray;">This is a paragraph.</p>
```

Listing 4: Inline CSS — avoid this in real projects

Problems with inline CSS:

- Hard to maintain — every element needs its own style
- Cannot reuse styles easily
- Mixes content and design (messy code)

Avoid inline CSS in real projects. Use it only for quick testing or special overrides.

2.2.2 2. Internal CSS

Written inside a `<style>` tag in the `<head>` section of the HTML file.

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <style>
5       h1 {
6         color: blue;
7         font-size: 30px;
8       }
9       p {
10        color: gray;
11      }
12    </style>
13  </head>
14  <body>
15    <h1>Welcome</h1>
16    <p>This is styled with internal CSS.</p>
17  </body>
18 </html>
```

Listing 5: Internal CSS — okay for small single-page projects

When to use: Small, single-page projects or quick demos. Not ideal for multi-page sites.

2.2.3 3. External CSS (MOST IMPORTANT — Recommended)

CSS is written in a **separate file** (e.g., `style.css`) and linked to HTML using `<link>`.

```
1 <!-- In index.html -->
2 <!DOCTYPE html>
3 <html>
4   <head>
```

```

5   <link rel="stylesheet" href="style.css">
6   </head>
7   <body>
8     <h1>Welcome</h1>
9     <p>Styled by external CSS.</p>
10  </body>
11 </html>

```

Listing 6: External CSS — the professional way

```

1  /* style.css */
2  h1 {
3    color: blue;
4    font-size: 30px;
5  }
6
7  p {
8    color: gray;
9    font-size: 16px;
10 }

```

Listing 7: style.css — separate file

Why external CSS is best:

- One CSS file can style all 100 pages of a website
- Changes in one file update the whole site
- Clean separation of HTML (structure) and CSS (design)
- CSS file is cached by the browser — pages load faster
- Industry standard — always use this method

Method	Where Written	Best For	Recommended?
Inline	Inside HTML tag	Emergency fixes	No
Internal	In <head> tag	Single-page demos	Sometimes
External	Separate .css file	All real projects	Yes — Always

2.3 CSS Selectors (Very Important)

Selectors decide *which* HTML element gets styled. This is one of the most important skills in CSS.

2.3.1 Element Selector

Targets all elements of a given HTML tag type.

```

1  /* Styles ALL <p> tags on the page */
2  p {

```



```
3   color: #333;
4   font-size: 16px;
5 }
6
7 /* Styles ALL <h1> tags */
8 h1 {
9   font-size: 32px;
10  color: navy;
11 }
```

Listing 8: Element selector — targets all matching tags

2.3.2 Class Selector

Targets elements with a specific `class` attribute. Uses a **dot (.)** prefix.

```
1  /* CSS */
2  .btn {
3    background-color: blue;
4    color: white;
5    padding: 10px 20px;
6    border-radius: 5px;
7  }
8
9  .card {
10   border: 1px solid #ccc;
11   padding: 20px;
12   border-radius: 8px;
13 }
```

Listing 9: Class selector — reusable, can apply to many elements

```
1  <!-- HTML -->
2  <button class="btn">Click Me</button>
3  <button class="btn">Submit</button> <!-- Same style reused -->
4  <div class="card">Card content here</div>
```

Listing 10: HTML using class selector

Rule: A class can be used on *multiple* elements. An element can also have *multiple* classes: `<div class="card highlight">`

2.3.3 ID Selector

Targets a **single unique** element with a specific `id`. Uses a **hash (#)** prefix.

```
1  /* CSS */
2  #navbar {
3    background-color: #222;
4    padding: 15px;
5  }
```

```
6
7 #hero-title {
8     font-size: 48px;
9     color: white;
10 }
```

Listing 11: ID selector — unique, use only once per page

```
1 <!-- HTML -->
2 <nav id="navbar">Navigation bar</nav>
3 <h1 id="hero-title">Welcome to My Site</h1>
```

Listing 12: HTML using ID selector

Class vs ID:

- Use **class** (.) for styles you will *reuse* on multiple elements
- Use **ID** (#) for a style applied to only *one unique* element per page
- In modern CSS, classes are preferred over IDs for styling

2.3.4 Group Selector

Apply the same styles to multiple selectors at once using a **comma** (,).

```
1 /* Without grouping --- repetitive */
2 h1 { color: navy; }
3 h2 { color: navy; }
4 h3 { color: navy; }
5
6 /* With grouping --- clean and efficient */
7 h1, h2, h3 {
8     color: navy;
9     font-family: Arial, sans-serif;
10 }
```

Listing 13: Group selector — avoid repeating the same rules

2.4 Colors in CSS

CSS supports multiple ways to specify colors.

```
1 /* 1. Named Colors --- easy to read, limited options */
2 h1 {
3     color: red;
4     background-color: white;
5 }
6
7 /* 2. Hex Codes --- most common, 6-digit code */
8 /* Format: #RRGGBB (R=Red, G=Green, B=Blue, each 00-FF) */
9 p {
```

```
10 color: #333333;      /* dark gray */
11 background-color: #f0f8ff; /* light blue */
12 }
13
14 h2 {
15   color: #e74c3c; /* a nice red */
16   color: #2ecc71; /* green */
17   color: #3498db; /* blue */
18 }
19
20 /* 3. RGB --- specify red, green, blue values (0-255) */
21 .box {
22   color: rgb(0, 0, 0);      /* black */
23   background-color: rgb(255, 255, 255); /* white */
24   color: rgb(52, 152, 219); /* blue */
25 }
26
27 /* Bonus: rgba --- adds alpha (transparency) 0=invisible, 1=solid */
28 .overlay {
29   background-color: rgba(0, 0, 0, 0.5); /* 50% transparent black */
30 }
```

Listing 14: Three ways to set colors in CSS

Quick Reference:

#000000 = Black #ffffff = White #ff0000 = Red
#00ff00 = Green #0000ff = Blue #ffff00 = Yellow

2.5 Text Styling

```
1 p {
2   color: #333333;      /* Text color */
3   font-size: 18px;     /* Size of the text */
4   font-family: Arial, sans-serif; /* Font type */
5   font-weight: bold;    /* Bold / normal / 100-900 */
6   text-align: center;   /* left | center | right | justify */
7 }
8
9 h1 {
10  font-family: 'Georgia', serif;
11  font-size: 36px;
12  font-weight: 700;
13  color: #1a1a2e;
14  text-align: left;
15  letter-spacing: 1px;   /* Space between letters */
16  line-height: 1.5;     /* Space between lines */
17  text-decoration: underline; /* underline | none | line-through */
18 }
```

Listing 15: Complete text styling example

Property	What it does
color	Sets the text color
font-size	Sets how big the text appears
font-family	Sets the typeface (Arial, Times, etc.)
font-weight	Sets thickness: normal, bold, or 100–900
text-align	Aligns text: left, right, center, justify
line-height	Sets vertical spacing between lines
letter-spacing	Adjusts spacing between individual letters
text-decoration	Adds underline, strikethrough, or removes them

3 CSS Units of Measurement (Very Important Core Topic)

Understanding CSS units is **essential**. Almost every CSS property that involves size — font-size, width, height, padding, margin — needs a unit.

3.1 Absolute Units

Absolute units have a **fixed, real-world size**. They do not change based on screen size or parent elements.

3.1.1 px — Pixels (MOST IMPORTANT)

px is the most common unit in CSS. One pixel is one dot on the screen.

```
1 h1 {
2   font-size: 32px;    /* 32 pixels tall */
3   border: 2px solid black; /* 2-pixel border */
4 }
5
6 .box {
7   width: 300px;
8   height: 200px;
9 }
```

Listing 16: Using pixels — most common absolute unit

When to use px: Borders, shadows, fixed-size icons, small UI details. Be careful using px for font-size — it overrides user browser settings.

3.1.2 Other Absolute Units (Brief Reference)

Unit	Full Name	Used For
cm	Centimeters	Print layouts
mm	Millimeters	Print layouts
in	Inches	Print / physical media

Note: cm, mm, in are rarely used in web CSS. They are meant for print stylesheets. Stick to px for screen layouts.

3.2 Relative Units (Very Important)

Relative units **scale based on something else** — the parent element, the root font size, or the screen size. This makes your design **flexible and responsive**.

3.2.1 % — Percentage

Percentage is **relative to the parent element's size**.

```
1 .container {
2   width: 800px;
3 }
4
5 .child {
6   width: 50%; /* 50% of 800px = 400px */
7 }
8
9 /* If parent changes to 600px, child becomes 300px automatically */
```

Listing 17: Percentage unit — relative to parent

Analogy: Think of “50%” like saying “take up half the space of your parent.”

3.2.2 em — Relative to Parent Font Size

em is relative to the **font-size of the parent element**. If no parent has a set font-size, it inherits from the browser default (usually 16px).

```
1 body {
2   font-size: 16px;
3 }
4
5 h1 {
6   font-size: 2em; /* 2 x 16px = 32px */
7 }
8
9 .container {
10  font-size: 20px;
11 }
12
13 .container p {
14  font-size: 1.5em; /* 1.5 x 20px = 30px --- not 1.5 x 16px! */
15 }
```

Listing 18: em unit — relative to parent font-size

The em problem — compounding: If a child inside a child uses em, the sizes multiply and become unpredictable. This is why **rem** was created.

3.2.3 rem — Relative to Root Font Size (Very Important)

rem stands for **Root EM**. It is **always** relative to the **font-size** of the `<html>` element (root), not the parent.

```
1 /* Set the root font size */
2 html {
3   font-size: 16px; /* 1rem = 16px everywhere */
```

```

4 }
5
6 h1 {
7   font-size: 2rem; /* 2 x 16px = 32px */
8 }
9
10 p {
11   font-size: 1rem; /* 1 x 16px = 16px */
12 }
13
14 .small-text {
15   font-size: 0.875rem; /* 0.875 x 16px = 14px */
16 }
17
18 /* Scale everything: change html font-size to 18px,
19    and ALL rem values update proportionally */

```

Listing 19: rem unit — always consistent, relative to html root

Why rem is preferred:

- Consistent — 1rem always equals the root font size
- No compounding problem like em
- Respects user browser font-size preferences (accessibility)
- Easy to scale the entire design by changing one value
- Modern CSS best practice: **use rem for font sizes, em for spacing within components**

3.2.4 vh and vw — Viewport Units

vh = Viewport Height and **vw** = Viewport Width.

100vh = the full height of the browser window. 100vw = the full width.

```

1 /* A full-screen hero section */
2 .hero {
3   height: 100vh; /* exactly the height of the browser window */
4   width: 100vw; /* exactly the width of the browser window */
5   background-color: #1a1a2e;
6 }
7
8 /* A section that takes half the screen height */
9 .half-screen {
10  height: 50vh;
11 }
12
13 /* Responsive font size based on screen width */
14 h1 {
15   font-size: 5vw; /* 5% of viewport width */
16 }

```

Listing 20: Viewport units — relative to browser window size

Analogy: 100vh is like saying “make this as tall as the whole window, no matter what screen size.”

3.3 When to Use Which Unit

Unit	Best Used For	Example
px	Borders, icons, fixed elements	border: 1px solid gray
%	Widths relative to parent	width: 80%
rem	Font sizes (consistent)	font-size: 1.25rem
em	Padding/margin inside components	padding: 1em
vh	Full-height sections, modals	height: 100vh
vw	Full-width banners	width: 100vw

```

1  html {
2    font-size: 16px;    /* Base: 1rem = 16px */
3  }
4
5  .card {
6    width: 50%;          /* 50% of parent width --- flexible */
7    padding: 1.5rem;     /* 24px --- scales with root font */
8    border: 1px solid #ddd; /* px is fine for borders */
9    border-radius: 8px;
10 }
11
12 .card h2 {
13   font-size: 1.5rem; /* 24px --- use rem for fonts */
14 }
15
16 .card p {
17   font-size: 1rem;    /* 16px */
18   line-height: 1.6;  /* unitless ratio --- best for line-height */
19 }
20
21 .hero {
22   height: 100vh;      /* Full screen height */
23 }

```

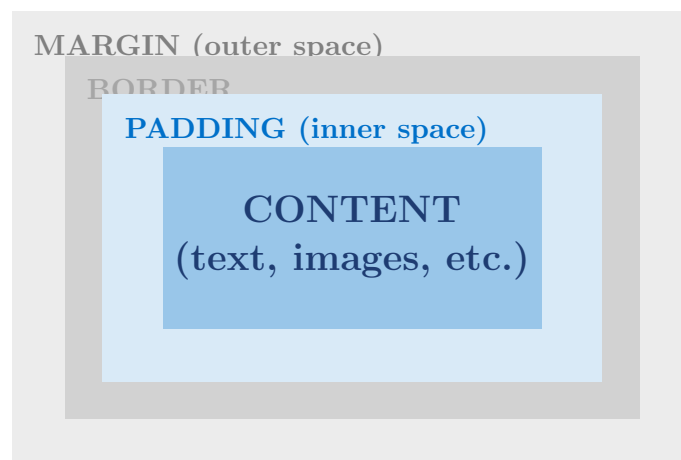
Listing 21: Real-world unit usage in a component

4 The Box Model (Core Concept — Must Understand)

The Box Model is one of the most important concepts in CSS. If you understand this, you can control spacing and layout of any element on the page.

4.1 What is the Box Model?

In CSS, every HTML element is a rectangular box. This box has four layers around it:



Real-world analogy: Think of a product in a box:

- **Content** = the actual product inside
- **Padding** = the foam/bubble wrap protecting the product inside the box
- **Border** = the walls of the cardboard box
- **Margin** = the empty space on the shelf between this box and the next box

4.2 The Four Parts Explained

```
1 .box {  
2   /* CONTENT */  
3   width: 300px;  
4   height: 150px;  
5  
6   /* PADDING --- space INSIDE the box, between content and border */  
7   padding: 20px;  
8   /* Or set each side individually: */  
9   /* padding-top: 10px; padding-right: 20px;  
10    padding-bottom: 10px; padding-left: 20px; */  
11  
12  /* BORDER --- the line around the box */
```

```

13 border: 3px solid navy;
14 /* border: [width] [style] [color] */
15 /* Styles: solid | dashed | dotted | double */
16
17 /* MARGIN --- space OUTSIDE the box, between this and other elements */
18 margin: 30px;
19 /* margin: auto; centers the box horizontally */
20 }

```

Listing 22: Box Model — all four properties

4.3 Total Element Size

By default, the total width of an element = width + padding (left+right) + border (left+right).

Example: width: 300px + padding: 20px (x2) + border: 3px (x2) = 346px total

This default can be confusing. The fix is `box-sizing: border-box`:

```

1  /* Best practice: apply to ALL elements */
2  * {
3    box-sizing: border-box;
4  }
5
6  /* Now: width: 300px INCLUDES padding and border.
7     The content shrinks to fit, but the total box stays 300px. */
8  .box {
9    width: 300px;
10   padding: 20px;
11   border: 3px solid navy;
12   /* Total width = exactly 300px */
13 }

```

Listing 23: box-sizing: border-box — makes sizing predictable

Always add `box-sizing: border-box` to your CSS reset. It makes layouts much easier to manage. Almost every professional project uses it.

4.4 Shorthand for Padding and Margin

```

1  /* 1 value: all four sides */
2  padding: 20px; /* top=20, right=20, bottom=20, left=20 */
3
4  /* 2 values: top/bottom left/right */
5  padding: 10px 20px; /* top+bottom=10, left+right=20 */
6
7  /* 4 values: top right bottom left (clockwise) */
8  padding: 10px 20px 15px 5px;
9

```

```

10 /* Same rules apply to margin */
11 margin: 0 auto; /* top/bottom=0, left/right=auto (centers block) */

```

Listing 24: Shorthand padding and margin — saves writing time

4.5 Display Property

The `display` property controls how an element behaves in the page flow.

```

1  /* BLOCK: takes full width, starts on new line */
2  /* Default for: div, h1-h6, p, ul, li, section */
3  div {
4      display: block;
5      width: 100%; /* takes full row */
6  }
7
8  /* INLINE: takes only as much width as content, NO new line */
9  /* Default for: span, a, strong, em, img */
10 span {
11     display: inline;
12     /* width and height have NO effect on inline elements */
13 }
14
15 /* INLINE-BLOCK: like inline (no new line) but width/height work */
16 .badge {
17     display: inline-block;
18     width: 80px;
19     height: 30px;
20     background-color: blue;
21     color: white;
22 }
23
24 /* NONE: hides the element completely (not visible, no space) */
25 .hidden {
26     display: none;
27 }

```

Listing 25: The three main display values

Value	New Line?	Width/Height?	Default Tags
block	Yes	Yes	div, p, h1, ul
inline	No	No	span, a, strong
inline-block	No	Yes	img, button
none	—	Hidden	—

5 Layout Systems

5.1 Flexbox (Must Learn — Modern CSS Essential)

Flexbox is the most important layout tool in modern CSS. Before Flexbox, centering a div vertically was notoriously difficult. Flexbox solves it in two lines.

5.1.1 What is Flexbox and Why Use It?

Flexbox (**Flexible Box Layout**) is a one-dimensional layout system. It arranges items in either a **row** or a **column** and gives you powerful control over alignment and spacing.

Real-world analogy: Imagine a shelf in a shop. Flexbox is like deciding how to arrange products on that shelf: stack them side by side? Space them evenly? Push them all to one end? Flexbox handles all of this.

When to use Flexbox:

- Navigation bars
- Centering elements (vertically and horizontally)
- Laying out a row of cards
- Any **one-dimensional** layout (either a row OR a column)

5.1.2 How Flexbox Works

Flexbox has a **container** and **items** inside it. You apply `display: flex` to the *container*. All direct children become *flex items*.

```
1  /* HTML Structure:
2     <div class="container">
3         <div class="item">A</div>
4         <div class="item">B</div>
5         <div class="item">C</div>
6     </div>
7  */
8
9  .container {
10     display: flex; /* Activates Flexbox */
11 }
12
13 /* By default: items are placed in a ROW (left to right) */
```

Listing 26: Basic Flexbox structure

5.1.3 Main Axis vs Cross Axis

- **Main axis:** the primary direction items are placed (*row* = horizontal, *column* = vertical)

- **Cross axis:** the perpendicular direction (if main = row, cross = vertical)

```
1 .container {  
2   display: flex;  
3   flex-direction: row;      /* DEFAULT: left to right */  
4   flex-direction: row-reverse; /* right to left */  
5   flex-direction: column;    /* top to bottom */  
6   flex-direction: column-reverse; /* bottom to top */  
7 }
```

Listing 27: flex-direction — sets the main axis

5.1.4 justify-content — Align Along Main Axis

```
1 .container {  
2   display: flex;  
3   justify-content: flex-start; /* items at start (DEFAULT) */  
4   justify-content: flex-end;  /* items at end */  
5   justify-content: center;    /* items centered */  
6   justify-content: space-between; /* max space BETWEEN items */  
7   justify-content: space-around; /* equal space AROUND each item */  
8   justify-content: space-evenly; /* equal space everywhere */  
9 }
```

Listing 28: justify-content — controls spacing along main axis

5.1.5 align-items — Align Along Cross Axis

```
1 .container {  
2   display: flex;  
3   align-items: stretch; /* items fill container height (DEFAULT) */  
4   align-items: flex-start; /* items at top */  
5   align-items: flex-end; /* items at bottom */  
6   align-items: center; /* items vertically centered */  
7 }
```

Listing 29: align-items — controls alignment on cross axis

5.1.6 Complete Flexbox Examples

```
1 /* Center a div both horizontally and vertically */  
2 .container {  
3   display: flex;  
4   justify-content: center; /* horizontal center */  
5   align-items: center; /* vertical center */  
6   height: 100vh; /* full screen height */  
7   background-color: #f0f0f0;  
8 }  
9  
10 .box {  
11   width: 200px;
```

```
12 height: 200px;
13 background-color: steelblue;
14 color: white;
15 }
```

Listing 30: Centering an element perfectly with Flexbox

```
1  /* HTML:
2     <nav class="navbar">
3       <div class="logo">MyBrand</div>
4       <ul class="nav-links">
5         <li><a href="#">Home</a></li>
6         <li><a href="#">About</a></li>
7         <li><a href="#">Contact</a></li>
8       </ul>
9     </nav>
10 */
11
12 .navbar {
13   display: flex;
14   justify-content: space-between; /* logo on left, links on right */
15   align-items: center;
16   padding: 1rem 2rem;
17   background-color: #1a1a2e;
18   color: white;
19 }
20
21 .nav-links {
22   display: flex;
23   gap: 20px; /* space between list items */
24   list-style: none;
25 }
```

Listing 31: Navigation bar using Flexbox

```
1 .card-container {
2   display: flex;
3   flex-direction: row;
4   gap: 20px;
5   flex-wrap: wrap; /* wrap items to next line if no space */
6 }
7
8 .card {
9   flex: 1; /* all cards grow equally */
10  min-width: 200px; /* minimum size before wrapping */
11  padding: 1.5rem;
12  border: 1px solid #ddd;
13  border-radius: 8px;
14 }
```

Listing 32: Card row using Flexbox

Flexbox Quick Reference:

- `display: flex` — turns the container into a flex container
- `flex-direction` — row (default) or column
- `justify-content` — controls spacing on the main axis
- `align-items` — controls alignment on the cross axis
- `gap` — sets space between items
- `flex-wrap: wrap` — allows items to wrap to next line

5.2 CSS Grid (Must Learn — 2D Layout System)

5.2.1 What is CSS Grid and Why Use It?

CSS Grid is a **two-dimensional layout system**. Unlike Flexbox (which handles one direction at a time), Grid lets you control *both rows and columns simultaneously*.

Real-world analogy: Think of a spreadsheet or a table. You have rows going across and columns going down. CSS Grid gives you the power to place items in specific cells, merge cells, and define the exact structure of your layout.

When to use Grid:

- Full page layouts (header, sidebar, main content, footer)
- Photo galleries
- Dashboard layouts
- Any **two-dimensional** layout (controlling rows AND columns)

5.2.2 Basic Grid Setup

```
1  /* HTML:
2     <div class="grid-container">
3         <div>Item 1</div>
4         <div>Item 2</div>
5         <div>Item 3</div>
6         <div>Item 4</div>
7     </div>
8  */
9
10 .grid-container {
11     display: grid; /* Activates Grid */
12 }
13 /* By default: all items stack in a single column */
```

Listing 33: Activating CSS Grid

5.2.3 grid-template-columns and grid-template-rows

```
1 .grid-container {
2   display: grid;
3
4   /* 3 equal columns */
5   grid-template-columns: 1fr 1fr 1fr;
6
7   /* Same as above using repeat() */
8   grid-template-columns: repeat(3, 1fr);
9
10  /* 3 columns: fixed 200px, flexible, fixed 200px */
11  grid-template-columns: 200px 1fr 200px;
12
13  /* 2 rows, each 100px tall */
14  grid-template-rows: 100px 100px;
15
16  /* Gap between grid items */
17  gap: 20px;           /* all sides */
18  column-gap: 20px;   /* columns only */
19  row-gap: 10px;      /* rows only */
20 }
```

Listing 34: Defining columns and rows in a Grid

What is fr?

The **fr** unit stands for **fraction** of the available space. `1fr 1fr 1fr` means: divide the space into 3 equal parts. `2fr 1fr` means: first column gets 2/3, second gets 1/3 of the space.

5.2.4 Complete Grid Examples

```
1 /* HTML:
2   <div class="gallery">
3     
4     
5     ... (6 images)
6   </div>
7 */
8
9 .gallery {
10  display: grid;
11  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
12  gap: 10px;                          /* 10px between images */
13 }
14
15 .gallery img {
16  width: 100%; /* fill column width */
17  height: 200px;
18  object-fit: cover; /* crop image to fit without distortion */
19  border-radius: 4px;
20 }
```


Listing 35: Photo gallery with CSS Grid

```

1  /* HTML:
2     <div class="page">
3       <header>Header</header>
4       <aside>Sidebar</aside>
5       <main>Main Content</main>
6       <footer>Footer</footer>
7     </div>
8  */
9
10 .page {
11   display: grid;
12   grid-template-columns: 250px 1fr; /* sidebar | main */
13   grid-template-rows: 80px 1fr 60px; /* header | content | footer */
14   min-height: 100vh;
15   gap: 0;
16 }
17
18 header { grid-column: 1 / 3; } /* span both columns */
19 aside { grid-column: 1; grid-row: 2; }
20 main { grid-column: 2; grid-row: 2; }
21 footer { grid-column: 1 / 3; } /* span both columns */

```

Listing 36: Full page layout with CSS Grid

Flexbox vs Grid — Simple Rule:

- **Flexbox** = 1D layout (a row of buttons, a column of items) — content drives the layout
- **Grid** = 2D layout (rows AND columns together) — you define the layout structure first
- They complement each other. Use Grid for the page structure, Flexbox for components inside.

5.3 Basic Layout Properties

```

1  .element {
2    /* Width and Height */
3    width: 400px;
4    height: 300px;
5    max-width: 1200px; /* never wider than 1200px */
6    min-width: 320px; /* never narrower than 320px */
7    min-height: 100vh; /* at least full viewport height */
8
9    /* Background */
10   background-color: #f5f5f5;
11   background-color: rgb(240, 240, 240);

```

```
12  
13  /* Center a block element horizontally */  
14  width: 80%;  
15  margin: 0 auto;    /* auto left/right margins = centered */  
16  }
```

Listing 37: Core layout properties

6 Learn-As-You-Go Topics (Introduction Only)

6.1 Position Property

The `position` property controls how an element is placed in the page flow.

```
1  /* STATIC --- default, follows normal document flow */
2  div { position: static; }
3
4  /* RELATIVE --- offset from its normal position, keeps its space */
5  .box {
6    position: relative;
7    top: 20px; /* moved 20px DOWN from where it would normally be */
8    left: 10px;
9  }
10
11 /* ABSOLUTE --- removed from flow, positioned relative to nearest
12    positioned ancestor (one that has position: relative/absolute/fixed) */
13 .tooltip {
14   position: absolute;
15   top: 0;
16   right: 0;
17   background: black;
18   color: white;
19   padding: 5px;
20 }
21
22 /* FIXED --- stays in place even when page is scrolled */
23 .navbar-sticky {
24   position: fixed;
25   top: 0;
26   left: 0;
27   width: 100%;
28   z-index: 100; /* appears above other content */
29 }
```

Listing 38: CSS position values

6.2 z-index

`z-index` controls the stacking order when elements overlap. Higher value = appears on top.

```
1  .modal-overlay {
2    position: fixed;
3    z-index: 1000; /* very high --- appears over everything */
4  }
5
6  .navbar {
```

```
7   position: fixed;
8   z-index: 100;
9 }
10
11 .content {
12   z-index: 1;      /* at the bottom of the stack */
13 }
14 /* z-index only works on elements with position != static */
```

Listing 39: z-index example

6.3 Pseudo-classes

Pseudo-classes style elements based on user interaction or state.

```
1  /* :hover --- styles applied when mouse moves over element */
2  .btn:hover {
3    background-color: darkblue;
4    cursor: pointer; /* shows hand cursor on hover */
5  }
6
7  a:hover {
8    text-decoration: underline;
9    color: blue;
10 }
11
12 /* :active --- styles applied while element is being clicked */
13 .btn:active {
14   background-color: navy;
15   transform: scale(0.98); /* slight shrink on click */
16 }
17
18 /* :focus --- styles when element is focused (e.g. clicked input) */
19 input:focus {
20   border-color: blue;
21   outline: none;
22 }
```

Listing 40: Common pseudo-classes

6.4 Transitions

Transitions create smooth animations when CSS properties change.

```
1  /* Basic syntax: transition: property duration timing-function */
2
3  .btn {
4    background-color: blue;
5    color: white;
6    padding: 10px 20px;
7    border-radius: 5px;
```

```
8   transition: background-color 0.3s ease; /* smooth color change */
9 }
10
11 .btn:hover {
12   background-color: darkblue; /* will transition smoothly over 0.3s */
13 }
14
15 /* Transition multiple properties */
16 .card {
17   transition: transform 0.2s ease, box-shadow 0.2s ease;
18 }
19
20 .card:hover {
21   transform: translateY(-4px); /* move up 4px */
22   box-shadow: 0 8px 20px rgba(0,0,0,0.15);
23 }
```

Listing 41: CSS transitions — smooth property changes

7 Common Mistakes to Avoid

1. Overusing Inline CSS

Using `style=""` on every element makes code impossible to maintain. Always move styles to an external CSS file. If you find yourself writing inline CSS for every element, stop and refactor.

2. Confusion Between `em` and `rem`

`em` is relative to the *parent* element's font-size and can compound unpredictably. `rem` is always relative to the root (`<html>`) font-size.

Rule: Use `rem` for font sizes. Use `em` for padding/margin inside a component.

3. Misunderstanding the Box Model

Forgetting that padding and border add to element width causes broken layouts. Always add `box-sizing: border-box` to your CSS reset to make sizing predictable.

4. Using Grid When Flexbox is Enough (or vice versa)

If you just need to lay out items in a single row or column, use Flexbox.

If you need to control rows AND columns at the same time, use Grid.

Do not use Grid for a simple horizontal navigation bar — that is Flexbox territory.

5. Forgetting `width: 100%` on images inside Grid/Flex

Images have a natural size. Inside a flex or grid container, add `width: 100%` and `object-fit: cover` to make them fill their container properly.

6. Not Understanding CSS Specificity

When two rules target the same element, the more *specific* one wins:

ID (`#`) $\succ$ Class (`.`) $\succ$ Element (tag)

If styles are not applying, check for specificity conflicts.

8 Best Practices

1. Always Use External CSS

Write all your styles in a separate `.css` file and link it with `<link>`. This keeps your HTML clean and your styles reusable and easy to maintain.

2. Start with a CSS Reset

Browsers apply their own default styles (margins, paddings) that vary by browser. Add a reset at the top:

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

3. Master Box Model and Layout First

Before learning animations or effects, deeply understand: Box Model (content, padding, border, margin), Flexbox, and Grid. These three concepts cover 80% of real-world CSS work.

4. Prefer Flexbox for 1D, Grid for 2D

Use Flexbox for components: navigation bars, card rows, button groups.

Use Grid for page-level layouts: header-sidebar-content-footer structures.

5. Use rem for Font Sizes

Set `font-size: 16px` on `html` and use `rem` everywhere else. This makes your typography consistent and accessible to users who change their browser font size.

6. Write Meaningful Class Names

Bad: `.div1`, `.box2`, `.blue-thing`

Good: `.navbar`, `.product-card`, `.submit-btn`

Class names should describe *what the element is*, not *how it looks*.

7. Organize CSS in Logical Order

Group related styles. A common order: reset → layout → components → utilities.

Comment your sections so others (and future you) can navigate the file.

9 Practice Exercises

Work through these exercises in order. Each builds on the previous concepts.

9.1 Exercise 1: Center a Div Using Flexbox

Goal: Create a blue box perfectly centered in the middle of the screen.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <link rel="stylesheet" href="style.css">
5 </head>
6 <body>
7   <div class="wrapper">
8     <div class="centered-box">I am centered!</div>
9   </div>
10 </body>
11 </html>
```

Listing 42: Exercise 1 — HTML

```
1 * {
2   margin: 0;
3   padding: 0;
4   box-sizing: border-box;
5 }
6
7 .wrapper {
8   display: flex;
9   justify-content: center;
10  align-items: center;
11  height: 100vh;
12  background-color: #f0f0f0;
13 }
14
15 .centered-box {
16   width: 200px;
17   height: 200px;
18   background-color: steelblue;
19   color: white;
20   font-size: 1rem;
21   display: flex;           /* center the text inside too */
22   justify-content: center;
23   align-items: center;
24   border-radius: 8px;
25 }
```

Listing 43: Exercise 1 — CSS (try to write this yourself first!)

9.2 Exercise 2: Create a Navbar Using Flexbox

Goal: Build a horizontal navigation bar with a logo on the left and links on the right.

```
1 <nav class="navbar">
2   <div class="logo">MyWebsite</div>
3   <ul class="nav-links">
4     <li><a href="#">Home</a></li>
5     <li><a href="#">Projects</a></li>
6     <li><a href="#">Blog</a></li>
7     <li><a href="#">Contact</a></li>
8   </ul>
9 </nav>
```

Listing 44: Exercise 2 — HTML

```
1 * { margin: 0; padding: 0; box-sizing: border-box; }
2
3 .navbar {
4   display: flex;
5   justify-content: space-between;
6   align-items: center;
7   background-color: #1a1a2e;
8   padding: 1rem 2rem;
9 }
10
11 .logo {
12   color: white;
13   font-size: 1.5rem;
14   font-weight: bold;
15 }
16
17 .nav-links {
18   display: flex;
19   list-style: none;
20   gap: 1.5rem;
21 }
22
23 .nav-links a {
24   color: #ccc;
25   text-decoration: none;
26   font-size: 1rem;
27   transition: color 0.2s ease;
28 }
29
30 .nav-links a:hover {
31   color: white;
32 }
```

Listing 45: Exercise 2 — CSS

9.3 Exercise 3: Create a Grid Photo Gallery

Goal: Display 6 images in a 3-column responsive grid.

```
1 <div class="gallery">
2   <div class="photo">Photo 1</div>
3   <div class="photo">Photo 2</div>
4   <div class="photo">Photo 3</div>
5   <div class="photo">Photo 4</div>
6   <div class="photo">Photo 5</div>
7   <div class="photo">Photo 6</div>
8 </div>
```

Listing 46: Exercise 3 — HTML

```
1 * { margin: 0; padding: 0; box-sizing: border-box; }
2
3 body {
4   padding: 2rem;
5   background-color: #f5f5f5;
6 }
7
8 .gallery {
9   display: grid;
10  grid-template-columns: repeat(3, 1fr);
11  gap: 16px;
12 }
13
14 .photo {
15   height: 200px;
16   background-color: steelblue;
17   color: white;
18   display: flex;
19   justify-content: center;
20   align-items: center;
21   font-size: 1.2rem;
22   border-radius: 6px;
23   transition: transform 0.2s ease;
24 }
25
26 .photo:hover {
27   transform: scale(1.03);
28 }
```

Listing 47: Exercise 3 — CSS

9.4 Exercise 4: Style a Profile Card

Goal: Build a clean profile card with a name, role, and button.

```
1 <div class="card-wrapper">
2   <div class="profile-card">
```

```
3   <div class="avatar">JD</div>
4   <h2 class="name">Jane Doe</h2>
5   <p class="role">Frontend Developer</p>
6   <p class="bio">
7     Passionate about building clean, accessible interfaces
8     and learning new CSS tricks every day.
9   </p>
10  <button class="follow-btn">Follow</button>
11 </div>
12 </div>
```

Listing 48: Exercise 4 — HTML

```
1 * { margin: 0; padding: 0; box-sizing: border-box; }
2
3 body {
4   font-family: Arial, sans-serif;
5   background-color: #e8edf2;
6 }
7
8 .card-wrapper {
9   display: flex;
10  justify-content: center;
11  align-items: center;
12  min-height: 100vh;
13 }
14
15 .profile-card {
16   background-color: white;
17   border-radius: 12px;
18   padding: 2rem;
19   width: 300px;
20   text-align: center;
21   box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
22 }
23
24 .avatar {
25   width: 80px;
26   height: 80px;
27   border-radius: 50%;
28   background-color: steelblue;
29   color: white;
30   font-size: 1.8rem;
31   font-weight: bold;
32   display: flex;
33   justify-content: center;
34   align-items: center;
35   margin: 0 auto 1rem;
36 }
37
38 .name {
```

```
39  font-size: 1.4rem;
40  color: #1a1a2e;
41  margin-bottom: 0.25rem;
42  }
43
44  .role {
45    color: steelblue;
46    font-size: 0.9rem;
47    margin-bottom: 1rem;
48  }
49
50  .bio {
51    color: #666;
52    font-size: 0.9rem;
53    line-height: 1.6;
54    margin-bottom: 1.5rem;
55  }
56
57  .follow-btn {
58    background-color: steelblue;
59    color: white;
60    border: none;
61    padding: 0.6rem 2rem;
62    border-radius: 20px;
63    font-size: 1rem;
64    cursor: pointer;
65    transition: background-color 0.2s ease;
66  }
67
68  .follow-btn:hover {
69    background-color: #1a5276;
70  }
```

Listing 49: Exercise 4 — CSS

10 Quick Reference Summary

10.1 CSS Units Cheat Sheet

Unit	Type	When to Use
px	Absolute	Borders, icons, fixed UI details
%	Relative (parent)	Fluid widths, responsive layouts
rem	Relative (root html)	Font sizes — always consistent
em	Relative (parent)	Padding/margin inside components
vh	Relative (viewport)	Hero sections, full-screen areas
vw	Relative (viewport)	Full-width banners, fluid typography
fr	Grid fraction	Grid column/row sizing

10.2 CSS Selectors Cheat Sheet

Selector	Syntax	Targets
Element	p { }	All <p> tags
Class	.card { }	All elements with class="card"
ID	#header { }	The element with id="header"
Group	h1, h2, p { }	All h1, h2, and p elements
Pseudo-class	a:hover { }	<a> when mouse hovers it
Universal	* { }	Every element on the page

10.3 Flexbox Cheat Sheet

Property	Values and Meaning
display	flex
flex-direction	row (default) column row-reverse column-reverse
justify-content	flex-start center flex-end space-between space-around
align-items	stretch (default) center flex-start flex-end
flex-wrap	nowrap (default) wrap
gap	spacing between items
flex	shorthand for grow, shrink, basis

10.4 Grid Cheat Sheet

Property	Values and Meaning
<code>display</code>	<code>grid</code>
<code>grid-template-columns</code>	<code>repeat(3, 1fr) 200px 1fr 200px</code>
<code>grid-template-rows</code>	<code>80px 1fr 60px</code>
<code>gap</code>	spacing between rows and columns
<code>grid-column</code>	<code>1 / 3</code> (span from line 1 to line 3)
<code>grid-row</code>	<code>2 / 4</code> (span row 2 to 4)

10.5 Box Model Cheat Sheet

Property	Meaning
<code>width / height</code>	Size of the content area
<code>padding</code>	Space inside the box (between content and border)
<code>border</code>	The visible line around the box
<code>margin</code>	Space outside the box (between this box and others)
<code>box-sizing: border-box</code>	width/height includes padding and border
<code>margin: 0 auto</code>	Horizontally centers a block element

11 Interview & Exam Revision Points

These are the most commonly asked questions in CSS interviews and exams. Know them well.

1. **What is the difference between class and ID selectors?**

Class (`.`) can be applied to multiple elements and reused. ID (`#`) must be unique — only one element per page should have a given ID.

2. **What are the three ways to add CSS? Which is best?**

Inline, Internal, External. **External CSS is best** — it separates concerns, is reusable, and is the industry standard.

3. **Explain the CSS Box Model.**

Every element is a box with four layers: **content** (the actual content), **padding** (space inside the box), **border** (the box edge), **margin** (space outside the box).

4. **What is the difference between `em` and `rem`?**

`em` is relative to the parent's font-size (can compound). `rem` is relative to the root `<html>` font-size (always consistent). Prefer `rem` for font sizes.

5. **What is `box-sizing: border-box`?**

It makes `width` and `height` include padding and border. This prevents unexpected layout overflow and is considered best practice.

6. **What is Flexbox and when do you use it?**

Flexbox is a 1D layout system. Use it for arranging items in a row or column: navbars, card rows, centering elements.

7. **What is CSS Grid and when do you use it?**

Grid is a 2D layout system. Use it when you need to control both rows and columns: page layouts, photo galleries, dashboards.

8. **How do you center a `div` both horizontally and vertically?**

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```

9. **What is the difference between `position: relative` and `absolute`?**

`relative` moves the element from its normal position but *keeps its space* in the layout. `absolute` removes the element from the flow and positions it relative to the nearest positioned ancestor.

10. **What is z-index?**

It controls the stacking order of overlapping elements. Higher z-index = appears on top. Only works on non-static positioned elements.

11. **What is the fr unit in CSS Grid?**

fr stands for fraction. It divides available space proportionally. `1fr 2fr` means the second column is twice as wide as the first.

12. **What is the difference between `display: none` and `visibility: hidden`?**

`display: none` removes the element completely (no space taken). `visibility: hidden` hides the element but *keeps its space* in the layout.

Final Note to Students

Your CSS Learning Path:

1. Understand **Box Model** completely — it underlies everything
2. Master **Flexbox** — you will use this every day
3. Learn **CSS Grid** — essential for real page layouts
4. Practice **CSS units** — know when to use rem, px, %, vh
5. Build real projects — style a portfolio page, a blog, a dashboard
6. Open browser DevTools — inspect and experiment with live CSS

CSS rewards hands-on practice. Do not just read — open a code editor, create HTML files, and type out every example in these notes. You will learn more in one hour of coding than in five hours of reading.

You now have everything you need to build beautiful web pages.